

app/components
CompareModal.tsx

```
app/components/CompareModal.tsx export default function CompareModal({ items, onClose, onRemove }: CompareModalP
88 88 >
89 89 { /* Remove button */}
90 90 <button
91 - onClick={() => onRemove(item.id)}
91 + onClick={() => onRemove(item.id)} /* Triggers deletion event from comparison indices */
92 92 className="absolute top-8 right-8 z-10 p-1.5 rounded-lg bg-black/60 text-white hover:bg-red-500 hover:text-white
    hover:scale-105 active:scale-95 transition-all cursor-pointer border border-white/10"
93 93 title="Hapus dari perbandingan"
94 94 >
```

prisma
schema.prisma

```
prisma/schema.prisma @@ -10,3 +10,18 @@ datasource db {
10 10 provider = "postgresql"
11 11 }
12 12
13 + enum DestinasiKategori {
14 + pantai
15 + wisata_alam
16 + wisata_kota
17 + heritage
18 + kuliner
19 + adventure
20 + }
21 +
22 + enum Role {
23 + admin
24 + editor
25 + guest
26 + }
27 +
```

prisma
schema.prisma

```
prisma/schema.prisma
@@ -39,3 +39,22 @@ model Destinasi {
  39 39     komentar      Komentar[]
  40 40  }
  41 41
  42 + model Komentar {
  43 +   id              Int           @id @default(autoincrement())
  44 +   destinasi        Destinasi @relation(fields: [destinasi_id], references: [id])
  45 +   destinasi_id     Int
  46 +   nama_user        String
  47 +   isi_komentar     String
  48 +   rating            Int
  49 +   created_at       DateTime @default(now())
  50 +
  51 +   @@index([destinasi_id])
  52 + }
  53 +
  54 + model User {
  55 +   id              Int           @id @default(autoincrement())
  56 +   username         String        @unique
  57 +   password          String
  58 +   role              Role         @default(admin)
  59 +   created_at       DateTime @default(now())
  60 + }
```

prisma
schema.prisma

```
prisma/schema.prisma
@@ -25,3 +25,17 @@ enum Role {
  25 25     guest
  26 26  }
  27 27
  28 + model Destinasi {
  29 +   id              Int           @id @default(autoincrement())
  30 +   nama            String
  31 +   lokasi          String
  32 +   deskripsi       String
  33 +   gambar_url      String?
  34 +   rating           Float         @default(0)
  35 +   is_viral         Boolean        @default(false)
  36 +   kategori         DestinasiKategori @default(pantai)
  37 +   created_at       DateTime       @default(now())
  38 +   updated_at       DateTime       @updatedAt
  39 +   komentar         Komentar[]
  40 + }
  41 +
```

- ▼  app
 - ▼  admin
 -  AdminDashboard.tsx
 - ▼  components
 -  AdminForm.tsx
 -  AdminTable.tsx
 -  page.tsx
 - ▼  api/destinasi
 -  route.tsx
 - ▼  components
 -  DetailModal.tsx
 -  Navbar.tsx
 - ▼  context
 -  ThemeContext.tsx
 - ▼  destinasi
 -  page.tsx
 - ▼  kontak
 -  page.tsx
 -  layout.tsx
 -  page.tsx

```

▼ app/admin/AdminDashboard.tsx
2  - import React, { useState } from 'react';
2  +
3  + import React, { useState, useEffect } from 'react';
3  4  import { Sun, Moon, Plus, Logout, LayoutDashboard, Map } from 'lucide-react';
4  5  import AdminTable from '../components/AdminTable';
5  6  import AdminForm from '../components/AdminForm';
7  + // 1. IMPORT USETHME GLOBAL DARI CONTEXT
8  + import { useTheme } from '@app/context/ThemeContext';
9  +
10 + // SINKRONISASI INTERFACE AGAR LENGKAP
11 + interface Destinasi {
12 +   id: number;
13 +   nama: string;
14 +   lokasi: string;
15 +   deskripsi: string;
16 +   gambar_url: string;
17 +   rating: number;
18 +   is_viral: boolean;
19 +   kategori: string;
20 + }
6  21
7  22   export default function AdminDashboard() {
8  -   const [theme, setTheme] = useState('dark');
23 +   // 2. KONSUMSI DATA TEMA GLOBAL (Ganti state theme lokal lama)
24 +   const { isDarkMode, setIsDarkMode } = useTheme();
25 +
26 +   // Ubah sebutan variabel biar kompatibel dengan kodingan bawah kamu
27 +   const theme = isDarkMode ? 'dark' : 'light';

```

Filter files...

- .gitignore
- package-lock.json
- prisma.config.ts
- prisma
- schema.prisma

Search within code

```
.gitignore
@@ -39,3 +39,5 @@ yarn-error.log*
39 39 # typescript
40 40 *.tsbuildinfo
41 41 next-env.d.ts
42 +
43 + /lib/generated/prisma
```

package-lock.json

Load Diff

Some generated files are not rendered by default. Learn more about [cush](#) [GitHub](#).

```
prisma.config.ts
*** @@ -0,0 +1,14 @@
1 + // This file was generated by Prisma, and assumes you have installed the following:
2 + // npm install --save-dev prisma dotenv
3 + import "dotenv/config";
4 + import { defineConfig } from "prisma/config";
5 +
6 + export default defineConfig({
7 +   schema: "prisma/schema.prisma",
8 +   migrations: {
```

Filter files...

- app/api
- auth
- login
- route.ts
- register
- route.ts
- contact
- route.tsx
- destinasi
- route.tsx
- komentar
- route.ts
- lib
- prisma.ts

Search within code

```
app/api/auth/login/route.ts
*** @@ -1,15 +1,21 @@
1 1 // app/api/auth/login/route.ts
2 2 import { NextResponse } from 'next/server';
3 - import { query } from '@lib/db';
3 + import { prisma } from '@lib/prisma';
4 4
5 5 export async function POST(request: Request) {
6 6   try {
7 7     const { username, password } = await request.json();
8 - const res = await query('SELECT * FROM users WHERE username = $1 AND password = $2', [username, password]);
9 8
10 - if (res.rows.length > 0) {
11 -   // Pastikan statusnya 200
12 -   return NextResponse.json({ message: 'Login Berhasil', user: res.rows[0] }, { status: 200 });
9 + if (!username || !password) {
10 +   return NextResponse.json({ message: 'Username dan password wajib diisi!' }, { status: 400 });
11 + }
12 +
13 + const user = await prisma.user.findUnique({
14 +   where: { username },
15 + });
16 +
17 + if (user && user.password === password) {
18 +   return NextResponse.json({ message: 'Login Berhasil', user }, { status: 200 });
13 19 } else {
14 20   return NextResponse.json({ message: 'Username atau Password salah' }, { status: 401 });
```

app
page.tsx

```
app/page.tsx
... @@ -1,15 +1,16 @@
1 1 "use client";
2 2
3 - import React, { useState, useEffect } from 'react';
3 + import React, { useEffect, useState } from 'react';
4 4 import Link from 'next/link';
5 5 import Navbar from '@app/components/Navbar';
6 + import { useTheme } from '@app/context/ThemeContext';
6 7 import {
7 8   Plane, Ship, ExternalLink, Mail,
8 9   Globe, Anchor, User, MessageSquare, Share2
9 10 } from 'lucide-react';
10 11
11 12 export default function PariwisataLampung() {
12 - const [isDarkMode, setIsDarkMode] = useState(true);
13 + const { isDarkMode } = useTheme();
13 14 const [mounted, setMounted] = useState(false);
14 15
15 16 useEffect(() => {
@@ -33,7 +34,7 @@ export default function PariwisataLampung() {
33 34 </div>
34 35
35 36 { /* Panggil Navbar Komponen */}
36 - <Navbar isDarkMode={isDarkMode} setIsDarkMode={setIsDarkMode} />
37 + <Navbar />
```

prisma/migrations
20260530161426_muktimig...
migration.sql
migration_lock.toml

```
prisma/migrations/20260530161426_muktimigrate/migration.sql
10 + "nama" TEXT NOT NULL,
11 + "lokasi" TEXT NOT NULL,
12 + "deskripsi" TEXT NOT NULL,
13 + "gambar_url" TEXT,
14 + "rating" DOUBLE PRECISION NOT NULL DEFAULT 0,
15 + "is_viral" BOOLEAN NOT NULL DEFAULT false,
16 + "kategori" "DestinasiKategori" NOT NULL DEFAULT 'pantai',
17 + "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
18 + "updated_at" TIMESTAMP(3) NOT NULL,
19 +
20 + CONSTRAINT "Destinasi_pkey" PRIMARY KEY ("id")
21 + );
22 +
23 + -- CreateTable
24 + CREATE TABLE "Komentar" (
25 + "id" SERIAL NOT NULL,
26 + "destinasi_id" INTEGER NOT NULL,
27 + "nama_user" TEXT NOT NULL,
28 + "isi_komentar" TEXT NOT NULL,
29 + "rating" INTEGER NOT NULL,
30 + "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
31 +
32 + CONSTRAINT "Komentar_pkey" PRIMARY KEY ("id")
33 + );
34 +
35 + -- CreateTable
36 + CREATE TABLE "User" (
37 + "id" SERIAL NOT NULL,
```

1. Gambar Pertama (Struktur Proyek dan .gitignore) Gambar ini menampilkan dua bagian utama yang sangat fundamental dalam memulai sebuah proyek pengembangan perangkat lunak, khususnya yang berbasis framework Next.js. Di sisi kiri, terdapat struktur pohon direktori (file tree) yang menunjukkan bagaimana proyek ini diorganisasikan. Kita dapat melihat adanya folder **app** yang merupakan inti dari routing pada Next.js, folder **public** untuk aset statis, serta berbagai file konfigurasi penting seperti **next.config.ts** dan **eslint**. Di sisi kanan, gambar menampilkan isi dari file **.gitignore**. File ini sangat krusial dalam sistem kontrol versi (seperti Git) karena fungsinya untuk memberitahu sistem mana saja file atau folder yang harus diabaikan (tidak dilacak dan tidak diunggah ke repositori). Terlihat aturan-aturan seperti mengabaikan folder **/node_modules** (karena folder ini berisi dependensi yang sangat besar dan bisa diinstal ulang), serta file-file hasil build produksi atau testing. Penggunaan **.gitignore** ini memastikan repositori tetap bersih, ringan, dan hanya berisi kode sumber inti serta konfigurasi yang diperlukan oleh developer lain, tanpa membawa sampah atau file yang bersifat lokal.

2. Gambar Kedua (Halaman Admin dan Logika Autentikasi) Gambar ini menampilkan lingkungan editor kode yang sedang mengembangkan halaman khusus admin, yakni file **app/admin/page.tsx**. Dari highlight kode yang ditampilkan, dapat diamati bahwa sedang dibangun sebuah fitur autentikasi (login dan registrasi) untuk masuk ke dashboard admin. Komponen ini menggunakan React Hook bernama **useState** untuk mengelola berbagai keadaan (state) dinamis pada form, seperti menampung inputan email, password, atau bahkan status apakah pengguna sedang dalam mode login atau registrasi. Selain itu, terlihat pula penggunaan logika **useEffect** untuk melakukan inisialisasi awal ketika komponen dirender, dan fungsi **handleSubmit** yang akan dijalankan ketika form disubmit. Fungsi submit ini mengirimkan data menggunakan metode POST melalui **fetch** ke backend API. Yang menarik perhatian adalah integrasi dengan library **Swal** (SweetAlert2), yang biasanya digunakan untuk menggantikan pop-up bawaan browser menjadi notifikasi yang jauh lebih menarik dan interaktif, misalnya untuk memberi peringatan jika login gagal atau berhasil.

3. Gambar Ketiga (Komponen Admin dan Koneksi API Prisma) Gambar ketiga menunjukkan dua file yang sedang dibuka secara bersamaan, menggambarkan pembagian tugas antara tampilan (frontend) dan server (backend). Pada file **app/admin/page.tsx** di bagian atas, terlihat kode yang mengimpor komponen **AdminDashboard** serta ikon-ikon dari **lucide-react** untuk memperindah antarmuka. Fungsi utama **AdminAuth** juga mendeklarasikan berbagai variabel state untuk mengontrol alur kerja halaman admin. Sementara itu, pada file **app/api/auth/komentar/route.ts** di bagian bawah, kita melihat sisi backend dari aplikasi Next.js. File ini mengimpor **NextResponse** untuk menangani respons HTTP dan mengimpor **Prisma**, yang merupakan sebuah Object-Relational Mapping (ORM). Penggunaan Prisma menandakan bahwa aplikasi ini berkomunikasi dengan database secara yang aman dan terstruktur. Rute API ini kemungkinan besar menangani proses verifikasi autentikasi sebelum memperbolehkan akses ke data komentar, memastikan hanya admin yang terautentikasi yang bisa melakukan operasi tersebut.

4. Gambar Keempat (Tabel Admin dan Konfirmasi Aksi) Pada gambar ini, fokusnya beralih ke file **app/admin/components/AdminTable.tsx**, yang merupakan komponen antarmuka

khusus untuk menampilkan data dalam bentuk tabel di halaman admin. Kode yang ditunjukkan menggarisbawahi implementasi konfirmasi aksi kritis, seperti menghapus data. Terlihat penggunaan properti seperti **cancelButtonColor** dan **confirmButtonText** yang merupakan bagian dari konfigurasi pop-up peringatan (kemungkinan lagi-lagi menggunakan SweetAlert). Terdapat struktur kondisional **if (confirm.isConfirmed)** yang memastikan sistem hanya akan menjalankan fungsi penghapusan data atau aksi berbahaya lainnya jika dan hanya jika admin secara sadar dan sengaja menekan tombol "Konfirmasi" pada dialog peringatan tersebut. Ini adalah praktik sangat baik dalam pengembangan User Experience (UX) untuk mencegah terjadinya penghapusan data yang tidak disengaja, memberikan lapisan keamanan tambahan pada operasi CRUD (Create, Read, Update, Delete).

5. Gambar Kelima (Perbandingan Perubahan Git dan Penghapusan File) Gambar kelima bukan menampilkan editor kode aktif, melainkan antarmuka dari sistem kontrol versi (seperti GitHub atau GitLab) yang menampilkan fitur "Diff" atau perbandingan perubahan kode. Di sisi kiri terdapat daftar file yang mengalami perubahan, termasuk **route.tsx** pada folder **api/contact** dan **package-lock.json**. Di sisi kanan, terdapat indikasi "This file was deleted" pada file rute API kontak, yang berarti pada commit atau versi ini, pengembang secara sengaja menghapus rute tersebut. Penghapusan ini bisa jadi karena fitur kontak dipindahkan ke layanan lain, di-refactor ke file berbeda, atau memang sudah tidak diperlukan lagi dalam arsitektur aplikasi. Adanya pesan tentang file **package-lock.json** yang tidak dirender karena merupakan file yang di-generate secara otomatis juga menunjukkan pemahaman bahwa file dependensi tidak perlu ditinjau secara manual dalam *code review* karena perubahannya mengikuti file **package.json**.

6. Gambar Keenam (State Management dan Sorting Data Destinasi) Gambar ini menampilkan file **page.tsx** (kemungkinan di bawah rute **destinasi**) yang sangat fokus pada pengelolaan state dan manipulasi data di sisi klien. Terlihat adanya deklarasi **useState** untuk berbagai kebutuhan interaktif pengguna, seperti **activeCategory** (untuk memfilter destinasi berdasarkan kategori), **showOnlyViral** (untuk menyaring destinasi yang sedang populer), **selectedDestinasi** (untuk menampung item yang sedang dipilih), dan **sortBy** (untuk menentukan kriteria pengurutan). Kode ini juga menampilkan fungsi asinkron **fetchDestinasi** yang bertugas mengambil data dari API. Bagian yang sangat menarik adalah logika pembentukan array **sortedDestinasi**, di mana terdapat serangkaian kondisi (if-else) yang memproses data mentah menjadi data yang sudah terurut secara dinamis. Pengguna dapat mengurutkan destinasi berdasarkan rating dari tertinggi ke terendah, atau berdasarkan nama secara alfabetis (ascending/descending), menunjukkan pengalaman pengguna yang sangat kaya dan interaktif.

7. Gambar Ketujuh (Definisi Tipe Data dan Halaman Destinasi) Masih berbicara tentang halaman destinasi, gambar ketujuh melengkapi gambar sebelumnya dengan menampilkan detail awal dari file **app/destinasi/page.tsx**. Hal paling menonjol di sini adalah penggunaan **interface Destinasi**. Dalam pengembangan modern menggunakan TypeScript, pendefinisian *interface* atau tipe data ini sangat penting karena bertindak sebagai "kontrak" atau cetak biru (blueprint) untuk objek data. Ini memastikan bahwa setiap objek destinasi yang beredar di dalam aplikasi memiliki properti yang konsisten (misalnya: id, nama, deskripsi, rating,

gambar), sehingga sangat mengurangi risiko *bug* akibat kesalahan penulisan nama properti atau tipe data yang tidak sesuai. Selain itu, terlihat juga impor ikon dari **lucide-react** yang akan digunakan sebagai elemen visual tombol atau indikator pada antarmuka halaman destinasi tersebut.

8. Gambar Kedelapan (Komponen Back to Top) Gambar ini menampilkan kode dari **app/components/BackToTop.tsx**, yang merupakan komponen kecil namun sangat penting untuk kenyamanan pengguna (UX). Komponen ini memanfaatkan **useState** untuk menyimpan status visibilitas (apakah tombol harus ditampilkan atau tidak) dan **useEffect** untuk melakukan side-effect. Di dalam **useEffect**, terdapat logika *event listener* yang mendengarkan peristiwa scroll pada jendela browser. Ketika pengguna menggulir halaman ke bawah melebihi ambang batas tertentu, state akan berubah dan tombol "Back to Top" akan muncul. Sebaliknya, jika halaman berada di posisi paling atas, tombol akan disembunyikan. Fungsi **scrollToTop** yang ada di dalamnya menggunakan metode **window.scrollTo** dengan perilaku *smooth* (halus), sehingga saat tombol diklik, halaman tidak langsung loncat ke atas secara kasar, melainkan bergulir dengan animasi yang elegan. Komponen ini juga mengimpor ikon **ArrowUp** dari **lucide-react** sebagai visual tombol.

9. Gambar Kesembilan (Modal Detail dan Fitur Berbagi) Pada gambar kesembilan, kita melihat isi dari **app/components/DetailModal.tsx**. Komponen ini bertugas menampilkan informasi rinci dari suatu item destinasi dalam bentuk jendela pop-up (modal). Kode yang ditampilkan menyoroti fitur canggih yang mengintegrasikan aplikasi dengan kemampuan bawaan perangkat atau browser, yaitu fitur berbagi (Share). Terdapat fungsi **handleShareModal** yang merangkai teks dan tautan (link) yang akan dibagikan. Yang sangat menarik adalah penggunaan **navigator.share**, yang merupakan Web Share API. API ini memungkinkan aplikasi web untuk memanggil dialog berbagi native/asli dari sistem operasi pengguna (misalnya dialog share di Android atau iOS), sehingga pengguna bisa langsung membagikan link destinasi ke WhatsApp, Telegram, Twitter, atau aplikasi pesan lainnya yang terinstal di perangkat mereka tanpa harus menyalin link secara manual.

10. Gambar Kesepuluh (Widget Informasi Cuaca) Gambar terakhir menampilkan file **app/components/WeatherWidget.tsx**, yang merupakan komponen tambahan (widget) untuk menampilkan informasi cuaca. Kemungkinan besar widget ini ditampilkan pada halaman detail destinasi untuk memberikan informasi prakiraan cuaca di lokasi wisata tersebut. Kode menunjukkan pembuatan **interface WeatherData** untuk memastikan data cuaca yang diterima dari API memiliki struktur yang jelas, seperti suhu, deskripsi cuaca, dan kelembapan. Terdapat juga **useState** untuk menyimpan data cuaca yang sudah diambil, serta

11. Gambar Pertama (Komponen CompareModal) Gambar ini menampilkan file **app/components/CompareModal.tsx** yang merupakan komponen antarmuka (UI) khusus untuk membandingkan beberapa destinasi wisata secara berdampingan. Penandaan **use client** di awal kode menunjukkan bahwa komponen ini berjalan di sisi klien (browser) karena membutuhkan interaktivitas. Terlihat adanya impor ikon dari **lucide-react** untuk elemen visual, serta **useTheme** untuk menyesuaikan tampilan dengan mode gelap atau terang. Hal paling krusial adalah pendefinisian dua *interface*: **Destinasi** yang memastikan setiap data

destinasi memiliki properti seperti **id** dan **nama** secara konsisten, serta **CompareModalProps** yang menjadi cetak biru bagi props komponen ini. Props tersebut menerima array **items** (daftar destinasi yang ingin dibandingkan), fungsi **onClose** untuk menutup modal, dan **onRemove** untuk mengeluarkan salah satu destinasi dari daftar perbandingan. Ini menunjukkan arsitektur React yang sangat rapi, di mana data dan aksi dikirim dari komponen induk, sementara komponen ini hanya bertugas merender tampilan perbandingannya.

12. Gambar Kedua (Halaman Kontak dan Data FAQ) Gambar ini menunjukkan file **app/kontak/page.tsx** yang sedang menyusun struktur data statis untuk halaman kontak atau informasi. Terlihat jelas impor berbagai ikon dari **lucide-react** seperti **Mail** (surat) dan **MapPin** (penanda lokasi) yang kelak akan digunakan sebagai visual pelengkap informasi kontak. Inti dari kode ini adalah pendefinisian array konstan bernama **FAQ_ITEMS**. Array ini menyimpan daftar objek di mana setiap objek berisi pasangan **question** dan **answer**. Contoh pertanyaan yang terlihat adalah "Kapan waktu terbaik untuk berkhijar ke Bandar Lampung?". Praktik memisahkan data teks panjang seperti FAQ ke dalam sebuah array terstruktur, alih-alih menuliskannya langsung secara hardcoded di dalam markup JSX, merupakan praktik terbaik (best practice). Hal ini membuat kode jauh lebih bersih, mudah dibaca, dan sangat memudahkan developer jika di masa depan ingin menambah, menghapus, atau mengambil data FAQ dari database secara dinamis.

13. Gambar Ketiga (Halaman Utama dan Manajemen State Global) Pada gambar ketiga, kita melihat file **app/page.tsx** yang merupakan halaman utama (homepage) dari aplikasi. Kode ini menunjukkan aktivitas impor yang sangat padat, mulai dari komponen navigasi (**Navbar**), manajemen tema (**useTheme**), hingga komponen interaktif seperti **WeatherWidget** dan **DetailModal**, yang menandakan halaman ini sangat kaya akan fitur. Fungsi utama **PariwisataLampung** mendeklarasikan berbagai state menggunakan React hooks, seperti **isDarkMode** untuk mengontrol tampilan gelap/terang, dan **mounted** untuk memastikan komponen hanya merender logika spesifik setelah halaman selesai dimuat di sisi klien (mencegah error hidrasi pada Next.js). Adanya penanda perubahan kode (highlight biru dan hijau) menunjukkan bahwa developer sedang aktif melakukan refactoring atau penambahan fitur pada halaman ini, mengintegrasikan berbagai komponen pendukung menjadi satu kesatuan pengalaman pengguna yang utuh.

14. Gambar Keempat (Backend API dan Transformasi Data Prisma) Gambar ini menampilkan tampilan perbandingan (diff) pada beberapa file backend, khususnya rute API komentar dan kontak, serta koneksi database **prisma.ts**. Fokus utama terletak pada proses transformasi dan pembersihan data sebelum dikirimkan ke frontend. Pada rute komentar, terlihat kode **result.map((k: any) => {...})** yang melakukan iterasi pada data mentah dari database. Proses ini (biasanya disebut mapping/transformasi) sangat penting untuk memformat ulang objek, misalnya mengubah format tanggal, menyembunyikan field sensitif, atau menambahkan field virtual. Hal yang sama terlihat pada rute kontak dengan **formatPesan**. Selain itu, terdapat logika penanganan nilai kosong atau null, seperti deklarasi **let email = "-"**, yang berfungsi sebagai *fallback* (nilai cadangan). Ini memastikan bahwa jika

data pengguna tidak lengkap di database, aplikasi tidak akan crash, melainkan menampilkan tanda strip yang rapi.

15. Gambar Kelima (Navbar dan Sinkronisasi LocalStorage) Gambar kelima mengungkap peningkatan fitur pada komponen **Navbar.tsx** yang berkaitan erat dengan pengalaman pengguna secara real-time. Perubahan kode yang ditandai dengan warna menunjukkan penambahan *hooks* **useState** dan **useEffect**. Secara spesifik, developer menambahkan state **favoritesCount** untuk melacak jumlah item favorit yang dimiliki pengguna, dengan nilai awal nol. Melalui **useEffect**, komponen ini membaca data dari **localStorage** (penyimpanan lokal browser) untuk menghitung berapa banyak destinasi yang telah ditandai sebagai favorit, lalu memperbarui state tersebut. Teknik ini memungkinkan ikon favorit di navbar langsung memperbarui angka notifikasinya secara instan tanpa harus melakukan refresh halaman atau memanggil API backend, menjadikan interaksi aplikasi terasa sangat cepat dan responsif.

16. Gambar Keenam (Penyimpanan Favorit dan Custom Event) Melengkapi gambar sebelumnya, gambar keenam menampilkan file **app/destinasi/page.tsx** yang berisi logika aksi ketika pengguna menekan tombol favorit. Fungsi **setFavorites** bertugas memperbarui state daftar favorit, yang kemudian diikuti oleh **localStorage.setItem** untuk menyimpan daftar favorit terbaru ke dalam memori browser agar data tidak hilang saat tab ditutup. Bagian paling canggih dari kode ini adalah pemanggilan **window.dispatchEvent(new Event('favorites-updated'))**. Ini adalah mekanisme Custom Event pada JavaScript. Ketika data favorit di halaman ini berubah, ia akan "meneriakkan" (dispatch) event khusus ke seluruh aplikasi. Navbar yang sedang "mendengarkan" (listen) event ini akan langsung menangkap sinyal dan mengupdate jumlah favoritnya. Ini adalah solusi elegan agar komponen yang tidak memiliki hubungan induk-anak (seperti halaman destinasi dan Navbar) bisa berkomunikasi secara reaktif.

17. Gambar Ketujuh (Navigasi Aktif dan Styling Dinamis) Masih berbicara tentang **Navbar.tsx**, gambar ketujuh memperlihatkan detail implementasi dari sisi tampilan navigasi. Di dalam fungsi render, terdapat logika kondisional yang memeriksa rute aktif, misalnya **pathname === '/destinasi'**. Jika pengguna sedang berada di halaman destinasi, maka link navigasi tersebut akan diberi kelas CSS khusus yang mencakup efek hover berwarna kuning (**hover:text-[#ffcc00]**), transisi halus (**transition-all**), dan garis bawah tebal (**border-b-2**). Penggunaan logika kondisional untuk styling ini sangat penting dalam UX (User Experience) karena memberikan umpan balik visual yang jelas kepada pengguna tentang di halaman mana mereka sedang berada saat ini, sehingga proses browsing menjadi lebih intuitif dan tidak membingungkan.

18. Gambar Kedelapan (DetailModal dan Drawer Berbagi) Gambar ini menampilkan versi yang lebih kompleks dari **DetailModal.tsx**. Selain impor standar, terlihat adanya integrasi dengan pustaka **Swal** (SweetAlert) untuk notifikasi, serta **@app/context/ThemeContext** untuk penyesuaian tema. Di dalam fungsi komponennya, dideklarasikan state seperti **showShareDrawer**. Ini mengindikasikan bahwa alih-alih langsung menggunakan fungsi berbagi bawaan sistem operasi, developer menerapkan pendekatan *Drawer* (panel yang meluncur dari bawah atau samping layar) yang berisi opsi-opsi berbagi. Drawer ini akan

menampilkan berbagai ikon media sosial atau metode salin tautan, sehingga memberikan kontrol yang lebih luas dan tampilan yang lebih estetik kepada pengguna sebelum mereka membagikan destinasi tersebut kepada orang lain.

19. Gambar Kesembilan (Integrasi Sosial Media dan Komentar) Melanjutkan DetailModal, gambar kesembilan memperlihatkan implementasi langsung dari logika yang dideklarasikan sebelumnya. Terdapat komentar-komentar penting seperti "// URL sharing social media" yang menandakan bagian kode tempat tautan dibuat dan disesuaikan untuk platform seperti WhatsApp, Twitter, atau Facebook. Kemudian, ada komentar "// Fetch komentar..." yang menunjukkan kode untuk melakukan pengambilan data ulasan atau komentar dari backend menggunakan fungsi asinkron. Pada bagian JSX-nya, terlihat elemen tombol dengan kelas gaya Tailwind seperti **rounded-xl bg-green-500**, yang merupakan tombol aksi berbagi atau tombol submit komentar dengan desain modern, berwarna, dan memiliki sudut membulat.

20. Gambar Kesepuluh (Fitur Pencarian Terkini dan Penyimpanan Lokal) Gambar terakhir menampilkan perbandingan kode pada **app/destinasi/page.tsx** yang berfokus pada fitur *search history* atau riwayat pencarian. Terlihat deklarasi state **recentSearches** untuk menyimpan daftar kata kunci yang baru saja dicari oleh pengguna, beserta state lain seperti **activeCategory**. Logika yang diimplementasikan melibatkan **localStorage**, di mana setiap kali pengguna melakukan pencarian, kata kunci tersebut akan disimpan ke penyimpanan lokal browser di dalam blok **try-catch**. Penggunaan **try-catch** di sini sangat bijak karena mengantisipasi kemungkinan error (misalnya jika browser pengguna dalam mode *Incognito* yang melarang penulisan ke **localStorage**), sehingga aplikasi tidak akan mengalami crash fatal. Fitur ini secara drastis meningkatkan kenyamanan pengguna, memungkinkan mereka mengklik ulang pencarian yang sering dilakukan tanpa harus mengetik ulang.

Gambar 21 (Baris 55–61): Keterangan Animasi Geser Halaman Ada penambahan komentar penjelasan di baris kode yang mengatur pergerakan halaman. Di situ dijelaskan bahwa fungsi tersebut digunakan untuk membuat efek animasi pergeseran layar yang halus (*smooth animated page scrolling*) saat mendeteksi elemen halaman bernama 'home'.

Gambar 22 (Baris 47–52): Dokumentasi Tombol Home Pengembang menambahkan blok komentar dokumentasi baru di atas fungsi **handleHomeClick**. Komentar ini menjelaskan bahwa fungsi tersebut bertugas untuk mengatur aksi ketika tombol 'Home' diklik, yaitu mengembalikan layar ke posisi paling atas atau mengalihkan pengguna kembali ke halaman utama (*landing zone*).

Gambar 23 (Baris 40–46): Keterangan Sinkronisasi Global Ditambahkan komentar penjelasan di bagian akhir fungsi efek. Catatan ini memperjelas bahwa sistem sedang mengaktifkan fungsi pengawas (*listener*) yang bertugas memantau perubahan data favorit secara global di seluruh aplikasi web.

Gambar 24 (Baris 38–44): Keterangan Sinkronisasi Awal Ditambahkan komentar penjelasan tepat di samping fungsi pembaruan favorit. Catatan ini menjelaskan bahwa

fungsi tersebut dijalankan untuk menyamakan (sinkronisasi) jumlah data favorit yang tersimpan sesaat setelah komponen navigasi berhasil dimuat pertama kali di layar.

Gambar 25 (Baris 14–20): Keterangan Pembacaan Tema (*Theme Context*)

Ditambahkan komentar penjelasan di samping fungsi tema global. Catatan tersebut menegaskan bahwa baris kode ini berfungsi untuk mengambil dan menggunakan status tema yang sedang aktif saat ini (apakah pengguna sedang memilih mode terang atau mode gelap).

Gambar 26 (Baris 17–23): Keterangan Pelacak Lokasi Favorit Ditambahkan komentar penjelasan di samping baris kode pembuatan status (*state*). Catatan baru ini menjelaskan bahwa baris tersebut berfungsi untuk melacak dan menyimpan jumlah lokasi wisata yang telah ditandai sebagai favorit oleh pengguna secara lokal.

Gambar 27 (Baris 11–17): Keterangan Pendeteksi Alamat Halaman Ditambahkan komentar penjelasan di baris paling atas di dalam fungsi komponen. Catatan ini memperjelas fungsi dari baris kode di bawahnya, yaitu untuk mengambil informasi mengenai alamat URL atau rute halaman web yang sedang dibuka oleh pengguna saat ini.

Gambar 28 (Baris 7–12): Dokumentasi Utama Komponen Navbar Ditambahkan blok komentar deskripsi utama di bagian atas berkas (sebelum fungsi komponen dimulai). Komentar ini menjelaskan tanggung jawab besar dari komponen Navbar ini, yaitu mengelola navigasi situs global, memetakan menu tautan, dan menyediakan tombol sakelar untuk mengubah tema halaman web.

Gambar 29 (Baris 249–264): Penataan Ulang Kotak Pencarian Gambar ini menunjukkan perubahan struktur tampilan pada kolom input pencarian. Kode lama yang menumpuk dihapus (ditandai warna merah) dan diganti dengan struktur baru (ditandai warna hijau). Tampilannya diubah menjadi susunan vertikal yang lebih rapi, lengkap dengan penataan posisi ikon kaca pembesar, teks petunjuk pencarian, serta penyesuaian warna latar belakang kotak agar otomatis berubah estetika saat pengguna berganti ke mode gelap (*isDarkMode*).

Gambar 30 (Baris 124–145): Penambahan Fitur Riwayat Pencarian Gambar terakhir ini menunjukkan penambahan dua fungsi logika baru yang cukup besar (ditandai blok hijau penuh). Fungsi pertama bertugas menyimpan kata kunci pencarian terakhir pengguna ke memori browser (*localStorage*), menyaring agar tidak ada kata yang ganda, dan membatasinya maksimal 5 riwayat saja. Fungsi kedua bertugas memberikan kemampuan bagi pengguna untuk menghapus atau membersihkan seluruh daftar riwayat pencarian tersebut.

Gambar 31 (Baris 300–316): Tombol untuk Menghapus Semua Riwayat Gambar ini menunjukkan penambahan tombol khusus bertuliskan "Hapus Semua" di pojok kanan area riwayat pencarian. Tombol ini diprogram agar ketika diklik, ia akan menjalankan fungsi untuk membersihkan seluruh daftar kata kunci yang pernah dicari pengguna dari memori browser (*localStorage*).

Gambar 32 (Baris 291–307): Judul Area Riwayat Pencarian Ditambahkan struktur tampilan teks untuk judul bagian riwayat pencarian dengan tulisan "Pencarian Terakhir". Pengembang mengatur ukuran teks, ketebalan, serta memberikan ikon jam kecil di sampingnya sebagai penanda visual bahwa area tersebut berisi daftar riwayat waktu lalu.

Gambar 33 (Baris 266–282): Logika Kondisional Area Riwayat Bagian ini menambahkan logika pengecekan di dalam tampilan (HTML/JSEX). Sistem diatur agar kotak riwayat pencarian hanya akan muncul (*conditional rendering*) jika kolom pencarian sedang aktif diklik (fokus) dan memori browser memang memiliki data riwayat pencarian yang pernah diketik sebelumnya.

Gambar 34 (Baris 333–349): Pengaturan Batas Maksimal Tampilan Gambar ini menunjukkan kelanjutan dari penutupan area riwayat pencarian. Di sini dipastikan bahwa daftar kata kunci yang muncul di layar akan dibatasi dan disusun rapi ke bawah agar tidak merusak tata letak elemen halaman lainnya.

Gambar 35 (Baris 315–331): Tombol Kata Kunci Riwayat Ditambahkan logika perulangan (*mapping*) untuk menampilkan setiap kata kunci dari riwayat pencarian dalam bentuk tombol-tombol kecil yang bisa diklik. Jika pengguna mengeklik salah satu tombol kata kunci tersebut, sistem akan otomatis mengisi kolom pencarian dengan kata tersebut dan langsung menyaring data destinasi wisata.

Gambar 36 (Baris 348–364): Penutupan Blok Riwayat Pencarian Gambar ini memperlihatkan penutupan tag HTML/JSEX dari seluruh kotak kontainer riwayat pencarian. Kode baru yang ditambahkan (warna hijau) memastikan bahwa struktur pembungkus area riwayat ini tertutup dengan benar tanpa merusak struktur kode di bawahnya.

Gambar 37 (Baris 363–379): Efek Animasi Transisi Ditambahkan pengaturan visual berupa efek transisi bayangan (*shadow*), tingkat keburaman latar belakang (*backdrop-blur*), dan animasi halus pada kotak riwayat pencarian. Hal ini dilakukan agar ketika kotak riwayat muncul atau menghilang, gerakannya terlihat estetik dan menyatu dengan tema halaman web.

Gambar 38 (Baris 359–375): Penyesuaian Responsif Layar Menunjukkan penyesuaian ukuran lebar kotak riwayat pencarian agar sifatnya responsif. Kotak ini diatur agar otomatis melebar penuh jika dibuka di layar HP (*w-full*), namun akan membatasi ukurannya sendiri jika dibuka melalui layar komputer/laptop (*lg:w-96*), mengikuti ukuran kolom input di atasnya.

Gambar 39 (Baris 373–389): Integrasi Mode Gelap untuk Kotak Riwayat Mirip dengan perubahan pada kolom input sebelumnya, baris kode ini mengatur warna latar belakang dan garis tepi dari kotak riwayat pencarian agar adaptif. Warnanya akan otomatis berubah menjadi putih transparan pada mode gelap (*isDarkMode*), dan menjadi putih bersih dengan bayangan lembut pada mode terang.

Gambar 40 (Baris 388–404): Batas Akhir Area Komponen Gambar terakhir ini menunjukkan baris penutup paling akhir dari seluruh rangkaian komponen visual pencarian baru ini. Baris ini menandakan bahwa proses penyisipan fitur antarmuka

(UI) untuk riwayat pencarian di dalam halaman destinasi telah selesai dan siap digunakan.

Gambar 41 (Baris 219–235): Fungsi Deteksi Klik di Luar (*Click Outside*) Gambar ini menunjukkan penambahan fungsi pengawas (*listener*) baru. Fungsi ini bertugas memantau setiap kali pengguna mengeklik layar komputer atau HP mereka. Jika sistem mendeteksi bahwa pengguna mengeklik di luar area kolom input atau kotak riwayat, maka kotak riwayat pencarian akan otomatis ditutup demi kenyamanan pengguna.

Gambar 42 (Baris 208–224): Pembuatan Referensi Komponen (*Ref Container*) Ditambahkan baris kode untuk membuat sebuah "jangkar" atau referensi (*searchRef*). Jangkar ini ditempelkan pada kotak pembungkus pencarian agar sistem tahu persis batas-batas area fisik dari kolom input dan kotak riwayat tersebut di layar.

Gambar 43 (Baris 199–215): Pemicu Penutupan Riwayat Menunjukkan logika lanjutan dari fungsi deteksi klik di luar. Jika kondisi terpenuhi (yaitu pengguna terbukti mengeklik area kosong di luar sistem pencarian), baris kode ini akan mengubah status keaktifan kolom pencarian menjadi mati (*false*), yang secara otomatis menyembunyikan kotak riwayat pencarian.

Gambar 44 (Baris 190–206): Pembersihan Pengawas Event (*Cleanup Listener*) Bagian ini menambahkan kode pembersih di dalam fungsi efek (*useEffect*). Tujuannya sangat penting untuk performa web: memastikan bahwa fungsi pengawas klik di luar tadi langsung dihapus dari memori browser saat pengguna pindah ke halaman lain, sehingga aplikasi tidak menjadi lemot.

Gambar 45 (Baris 181–197): Pemasangan Pengawas Event (*Mount Listener*) Kebalikan dari gambar sebelumnya, baris kode di gambar ini berfungsi untuk mendaftarkan atau mengaktifkan fungsi pengawas klik (*click event listener*) ke dalam sistem operasi browser sesaat setelah halaman destinasi selesai dimuat.

Gambar 46 (Baris 172–188): Ketergantungan Fungsi Efek (*Dependency Array*) Gambar ini memperlihatkan batas penutup dari fungsi efek beserta daftar ketergantungannya (*dependencies*). Kode ini memastikan bahwa fungsi pemantau klik di luar hanya dibuat satu kali saja di awal dan tidak berjalan berulang-ulang tanpa arah.

Gambar 47 (Baris 157–173): Pengaturan Aksi Saat Kolom Diklik (*On Focus*) Ditambahkan fungsi pemicu baru pada kolom input. Ketika pengguna mengeklik atau memindahkan kursor ke dalam kotak pencarian (*onFocus*), sistem akan langsung mengubah status kolom menjadi aktif (*true*). Aksi inilah yang memicu munculnya daftar riwayat pencarian terakhir di bawah kolom.

Gambar 48 (Baris 146–162): Menempelkan Referensi pada Komponen Utama Baris kode ini menunjukkan proses penempelan jangkar referensi (*searchRef*) yang sudah dibuat pada Gambar 2 ke dalam tag pembungkus visual (*div*) terluar dari seluruh sistem pencarian.

Gambar 49 (Baris 135–151): Pengaturan Aksi Saat Tombol Riwayat Diklik

Ditambahkan logika di mana jika pengguna mengeklik salah satu kata kunci di dalam daftar riwayat pencarian, sistem tidak hanya mengisi kolom teks dan menyaring data destinasi, tetapi juga otomatis mengubah status fokus kolom menjadi mati (*false*) agar kotak riwayat langsung menutup setelah pilihan diklik.

Gambar 50 (Baris 124–140): Sinkronisasi Penutupan Setelah Aksi Berhasil

Gambar terakhir ini memastikan sinkronisasi akhir dari alur interaksi. Baris kode ini memastikan bahwa setelah pengguna menekan tombol "Hapus Semua" atau memilih salah satu riwayat pencarian, status tampilan antarmuka akan langsung diperbarui dengan bersih dan kotak riwayat menutup dengan rapi tanpa ada bagian yang macet di layar.

Gambar 51 (Baris 82–98): Inisialisasi Riwayat dari Penyimpanan Browser

Gambar ini menunjukkan pembuatan status (*state*) baru bernama *recentSearches*. Status ini diatur agar saat halaman web pertama kali dimuat, sistem akan langsung memeriksa dan mengambil daftar riwayat pencarian lama yang tersimpan di dalam memori lokal browser (*localStorage*).

Gambar 52 (Baris 72–88): Status Pengawas Kolom Input Ditambahkan sebuah status pelacak baru bernama *isFocused*. Fungsi dari status ini sangat penting untuk antarmuka, yaitu sebagai sakelar digital untuk memantau apakah pengguna sedang mengeklik aktif di dalam kotak pencarian atau tidak.

Gambar 53 (Baris 62–78): Pembuatan Jangkar Referensi (*Search Ref*)

Menunjukkan baris kode pembuatan jangkar referensi (*useRef*) yang sempat kita bahas pada kelompok gambar sebelumnya. Di sini baris tersebut baru diinisialisasi secara resmi agar nantinya bisa ditempelkan pada fisik kotak kontainer pencarian.

Gambar 54 (Baris 51–67): Integrasi Fungsi Bawaan React Pengembang menambahkan fungsi pengait bawaan dari React, yaitu *useRef*, ke dalam daftar fungsi yang digunakan di halaman ini. Fungsi ini sangat dibutuhkan untuk mendukung logika deteksi klik di luar area pencarian yang kita bahas sebelumnya.

Gambar 55 (Baris 40–56): Batas Atas Fungsi Utama Halaman Gambar ini memperlihatkan area bagian atas di dalam fungsi utama halaman destinasi. Kode baru yang ditandai warna hijau menunjukkan peletakan awal baris-baris logika baru (seperti pembuatan status dan referensi) tepat setelah komponen halaman tersebut mulai dijalankan.

Gambar 56 (Baris 248–264): Pengaturan Tombol Tekan pada Kolom Input

Ditambahkan fungsi pemicu baru pada kolom teks pencarian, yaitu *onKeyDown*. Fungsi ini bertugas mengawasi tombol kibor yang ditekan oleh pengguna saat mereka sedang mengetik di dalam kotak pencarian.

Gambar 57 (Baris 238–254): Deteksi Tombol Enter Melanjutkan fungsi pengawas kibor dari gambar sebelumnya, baris kode ini mendeteksi secara spesifik jika pengguna menekan tombol **Enter**. Jika tombol tersebut ditekan, sistem akan otomatis menjalankan fungsi untuk menyimpan kata kunci tersebut ke dalam riwayat pencarian terakhir.

Gambar 58 (Baris 228–244): Pengaman Logika Tombol Enter Gambar ini menunjukkan bagian akhir penutupan dari logika deteksi tombol Enter pada kolom input pencarian. Kode ini memastikan bahwa proses penyimpanan riwayat hanya dipicu oleh tombol Enter dan tidak terganggu oleh tombol kibor lainnya.

Gambar 59 (Baris 218–234): Penyelarasan Alur Pengetikan Menunjukkan penyelarasan posisi kode setelah fungsi deteksi klik di luar dipasangkan. Hal ini memastikan alur interaksi pengguna (antara mengetik teks, menekan Enter, atau mengeklik area kosong di layar) dapat berjalan beriringan tanpa bentrok.

Gambar 60 (Baris 208–224): Batas Akhir Fungsi Pengawas Klik Gambar terakhir ini menutup rangkaian fungsi pengawas klik di luar area kontainer pencarian. Baris ini menandakan bahwa seluruh logika pengaman antarmuka untuk memunculkan dan menyembunyikan riwayat pencarian telah terpasang secara utuh dan terstruktur dengan rapi.

Gambar 61 (Baris 131–147): Penghapusan Fungsi Pencarian Lama Gambar ini menunjukkan penghapusan baris kode lama yang ditandai dengan warna merah. Pengembang menghapus fungsi pencarian atau penanganan teks standar yang lama karena logikanya sudah digantikan oleh fungsi baru yang lebih pintar (yang sudah mendukung penyimpanan riwayat pencarian otomatis maksimal 5 kata kunci).

Gambar 62 (Baris 114–130): Otomatisasi Penutupan Kotak Riwayat Ditambahkan baris kode baru di dalam fungsi penanganan klik riwayat. Ketika pengguna mengeklik salah satu kata kunci dari daftar riwayat pencarian terakhir, selain kolom teks otomatis terisi, sistem langsung mengubah status fokus menjadi mati (`setIsFocused(false)`). Hal ini memastikan kotak riwayat langsung menutup secara instan setelah kata kunci dipilih.

Gambar 63 (Baris 105–121): Pengosongan Status Setelah Riwayat Dihapus Gambar ini menunjukkan logika penutup dari fungsi "Hapus Semua". Ditambahkan kode pengaman agar setelah memori browser dikosongkan dari riwayat pencarian, status riwayat di dalam aplikasi juga langsung diubah menjadi susunan kosong (`[]`) dan kotak riwayat otomatis menutup secara rapi.

Gambar 64 (Baris 92–108): Struktur Baru Fungsi Penyimpanan Riwayat Menunjukkan penataan ulang posisi fungsi penambahan riwayat pencarian baru (`saveSearch`). Kode ini dipindahkan posisinya ke bagian atas agar urutan pembacaan logika kode dari atas ke bawah (inisialisasi status memori, fungsi hapus, baru fungsi simpan) menjadi lebih terstruktur dan mudah dirawat.

Gambar 65 (Baris 82–98): Batas Penutup Inisialisasi Riwayat Gambar terakhir ini memperlihatkan batas penutup dari proses pengambilan data awal riwayat dari memori browser (`localStorage`). Kode ini memastikan bahwa jika di dalam browser pengguna belum ada riwayat pencarian sama sekali, aplikasi tidak akan eror dan otomatis menyiapkan daftar kosong sebagai gantinya.

